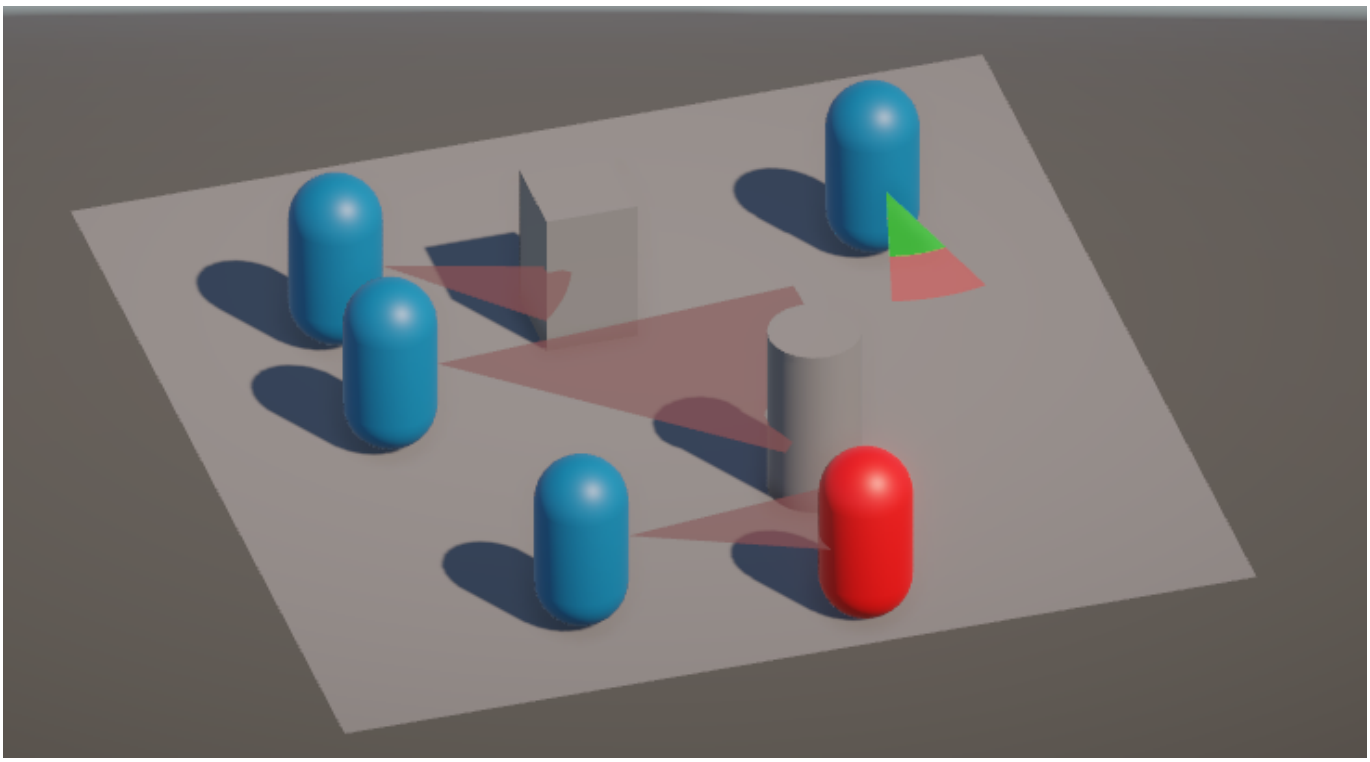
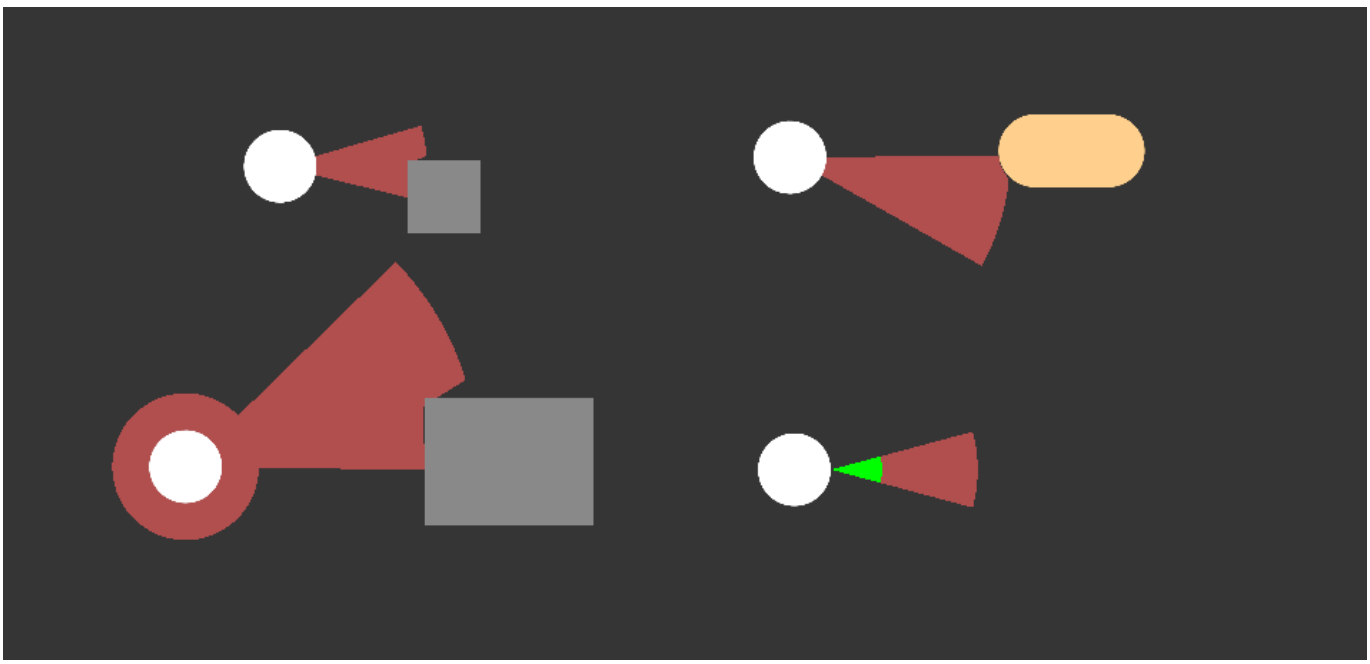
SimpleFOV Documentation

Overview

SimpleFOV is a small Unity tool that can help you create a field of view with the ability to reshape by obstacles and detect objects in both 2D and 3D games. By adding a few required components and script, it would be easy to start to use it in your own project, and there are examples to refer to in all kinds of project templates.





Note that this tool is based on raycasting, so it is not ideal for use case that needs too many such FOVs. It should be handy for use cases like stealth game.

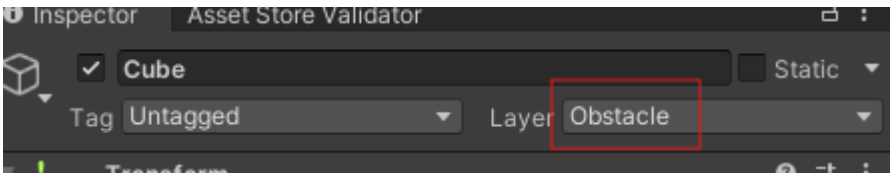
The 3D and 2D implementation is mostly the same despite a few changes and different supported ways of handling object detection within the FOV. You can go directly to the 3D/2D section that is related to your question.

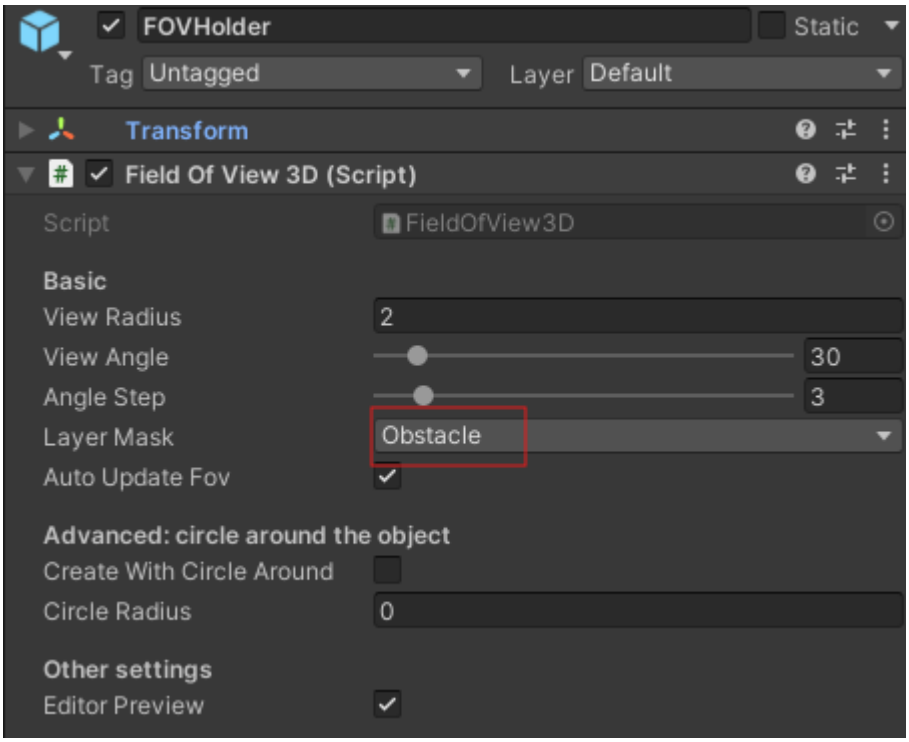
If you have any question, feel free to reach out to me via email: qlgamedev@gmail.com

Easiest way to get started is to refer to examples. If you prefer, you can also see videos of setting up from scratch: 3D https://www.youtube.com/watch?v=OVxMkD0Sjfc 2D https://www.youtube.com/watch?v=mnZltwFV3Ls

Examples

In both 3D and 2D, there are three examples in the scene. The only setup you need to do is to set up the layer of obstacles to be a new-added layer, for example "Obstacle". Then set the "layer mask" field in the Field of View component to be that layer.





"ExampleBot" shows a basic usage of the field of view functionality, whose field of view shape updates according to the obstacles.

"ExampleBot - with detection" shows how to detect object going into the FOV area. Due to some limitations in 3D side, this part is implemented differently in this tool.

"ExampleSkillRelease" simply uses the field of view to create a sector with growing radius. It turns off the "Auto Update FOV" and call the FOV update itself, as an example to control the update yourself instead of automatically update every frame.

3D

Get Started

The example shows how to use this tool in some common cases. If you want to start from zero, here is the guide.

(1) Choose an object to hold the field of view, preferrably an empty child object of your game object.

(2) Attach the script "FieldOfView3D.cs" to the object.

(3) Add a Unity "Mesh Renderer" component to the object and assign a suitable material to the Materials field. You might want to turn off "Cast Shadows".

(4) Add a Unity "Mesh filter" component to the object.

(5) Play the game and adjust the FieldOfView3D parameters to fit your need.

(6) For objects that should block the FOV, pick a specific layer for them. In the FieldOfView3D component attached to your object, set the "Layer Mask" parameter to that layer. All objects in that layer will block FOV.

(7) If you need to detect something going into the fov area, we have unity events and public properties available for this. We differentiate between blocker objects (layers you set in the "layerMask", which affects fov shape), and non-blocker objects (layers you set in the "nonBlockerLayerMask", which ideally is different from "layerMask" because non-blocker detection runs an extra raycasting, so it's bad for performance.).

FieldOfView3D script

Inspector

viewRadius The radius of the field of view.

viewAngle The angle of the view. Will take the transform right vector as the center line to display the field of view.

angleStep The angle step for one triangle (the circle or sector shape is formed by many triangles). Decrese the value to get a more good-looking sector/circle, but it affects performance.

layerMask The layer mask that fov should detect the obstacle using the raycast. The fov is blocked by the obstacle.

autoUpdateFov Automatically update fov. Turn this off to control the update yourself by calling UpdateFOV() or if you don't need to update the fov (usually you don't need to update the fov if you don't have any obstacles blocking the fov).

createWithCircleAround Turn this on to also have a circle around the object. (Usually it might not be so useful in 3D game.)

circleRadius The radius of the circle around the object. This is only effective if createWithCircleAround is on.

shouldDetectNonBlockers Collect all the objects detected including non-blocker objects when scanning. Enabling this costs extra time runtime.

nonBlockerLayerMask The layers which need to be detected as non-blocker. Note that if you include the blocker layer mask (defined in layerMask field), it will still report the object even though it's a blocker.

editorPreview Enable this to show brief lines representing the fov area in editor scene. Only simple lines are presented in the editor to serve as a direction preview for ease-of-adjustment in editor.

BlockerDetected UnityEvent<GameObject, GameObject, Vector3> triggered when blocker object is detected. Triggered per detected object, at max once per detected object per scan. Arguments: fov3D holder game object, detected game object, detected point

NonBlockerDetected Same to the above except for non-blockers (need set up shouldDetectNonBlockers and nonBlockerLayerMask).

Public properties and methods

UpdateFOV() Call this in your script to control the update of the FOV yourself. Remember to turn off **autoUpdateFov**.

DetectedBlockersList IReadOnlyList a list of detected blocker objects information in the latest scan

DetectedNonBlockersList Same as above except for non-blockers

Feel free to expose more private variables if needed, like I need "ViewRadius" myself so I exposed it.

A bit more words on the detection in 3D. First, I wanted to make it to use mesh collider component, and did make it to extend the fov shape in y-axis to be a collider with height that can detect collision/trigger with other objects. However, Unity does not support collision detection for non-convex mesh collider. Unfortunately, when there are obstacles, the collider will not be convex most of the time, so the collider cannot be correctly updated to the exact shape of the fov shape, even if the sharedMesh for the collider is correctly the fov shape extended in y-axis a bit. If you feel like this tryout would be useful for you, you are welcome to contact me to get that code to test.

2D

Get Started

The example shows how to use this tool in some common cases. If you want to start from zero, here is the guide.

(1) Choose an object to hold the field of view, preferrably an empty child object of your game object.

(2) Attach the script "FieldOfView2D.cs" to the object.

(3) Add a Unity "Mesh Renderer" component to the object and assign a suitable material to the Materials field. You might want to turn off "Cast Shadows".

(4) Add a Unity "Mesh filter" component to the object.

(5) Play the game and adjust the FieldOfView2D parameters to fit your need.

(6) For objects that should block the FOV, pick a specific layer for them. In the FieldOfView2D component attached to your object, set the "Layer Mask" parameter to that layer. All objects in that layer will block FOV.

(7) If you need to detect something going into the fov area, add a "Polygon Collider 2D" component to the object that holds the FOV2D script. Turn on the "Should Update Collider"

setting in FieldOfView2D component. The collider shape will be updated to be the same shape as the visible FOV shape.

FieldOfView2D script

viewRadius The radius of the field of view.

viewAngle The angle of the view. Will take the transform right vector as the center line to display the field of view.

angleStep The angle step for one triangle (the circle or sector shape is formed by many triangles). Decrese the value to get a more good-looking sector/circle, but it affects performance.

layerMask The layer mask that fov should detect the obstacle using the raycast. The fov is blocked by the obstacle.

autoUpdateFov Automatically update fov. Turn this off to control the update yourself by calling UpdateFOV() or if you don't need to update the fov (usually you don't need to update the fov if you don't have any obstacles blocking the fov).

createWithCircleAround Turn this on to also have a circle around the object. This can be sometimes useful in 2D stealth game, where you might want a circle-like area around the enemy to also detect the player.

circleRadius The radius of the circle around the object. This is only effective if createWithCircleAround is on.

editorPreview Enable this to show brief lines representing the fov area in editor scene. Only simple lines are presented in the editor to serve as a direction preview for ease-of-adjustment in editor.

shouldUpdateCollider If you want to detect collision, turn on this, and remember to add a PolygonCollider2D component to the same object that has the FieldOfView2D component.

UpdateFOV() Call this in your script to control the update of the FOV yourself. Remember to turn off **autoUpdateFov**.